

DOCUMENTO DE ARQUITETURA

E-Commerce de Camisetas

Diagramas C4 · Diagrama de Classes · Padrões de Projeto

Stack	Java 21 + Spring Boot 3.x + Apache Kafka + Docker
Curso	Análise e Desenvolvimento de Sistemas — 4º Período
Semestre	2026/1

Sumário

1.	Visão Geral da Arquitetura	3
2.	Diagrama C4 — Nível Contexto	4
3.	Diagrama C4 — Nível Container	5
4.	Diagrama de Classes do Domínio	6
5.	Fluxo Assíncrono Kafka	7
6.	Design Patterns Aplicados	8
7.	Java 21 — Features Utilizadas	9
8.	Estrutura dos Repositórios	10

1. Visão Geral da Arquitetura

O projeto é um e-commerce de camisetas construído sobre uma arquitetura de microsserviços. Cada serviço possui responsabilidade única, banco de dados próprio e comunicação assíncrona via Apache Kafka. Um API Gateway centraliza o roteamento. Todo o ambiente é containerizado com Docker Compose.

Princípio central: o order-service não sabe que o notification-service existe. Ele publica um evento no Kafka e qualquer serviço interessado pode consumi-lo. Isso é o desacoplamento exigido pelo documento norteador (0,4 pts na N2).

Stack Tecnológico

Camada	Tecnologia
Backend	Java 21 + Spring Boot 3.x (Spring Web, Data JPA, Kafka, WebSocket)
Mensageria	Apache Kafka 7.5 + Zookeeper (Docker Compose)
Banco de Dados	PostgreSQL 16 (um por serviço) + Redis 7 (cache de status)
Gateway	Spring Cloud Gateway (roteamento, CORS, rate limit)
Frontend	React + Vite (WebSocket via STOMP/SockJS)
Infraestrutura	Docker Compose (todos os containers locais)
Testes	JUnit 5 + Mockito + Testcontainers (PostgreSQL + Kafka)

2. Diagrama C4 — Nível Contexto

O nível Contexto do modelo C4 mostra o sistema como uma caixa preta e seus relacionamentos com os atores externos. Identifica quem usa o sistema e com quais sistemas externos ele se comunica.



Figura 1 — Diagrama C4 Nível Contexto: sistema e atores externos

Atores e Sistemas Externos

Ator / Sistema	Responsabilidade
Cliente (Comprador)	Pessoa física que navega na vitrine, adiciona ao carrinho e acompanha o pedido em tempo real.
Administrador	Responsável pela loja: cadastra/edita produtos, visualiza pedidos e acessa o dashboard.
Apache Kafka	Broker de mensagens externo ao domínio de negócio. Recebe eventos do order-service e os distribui.
E-mail / Push	Serviço externo de notificação (ex: SendGrid). Acionado pelo notification-service para alertar o cliente.

3. Diagrama C4 — Nível Container

O nível Container detalha os processos em execução dentro do sistema: os microsserviços Spring Boot, o API Gateway, os bancos de dados e o broker Kafka. Cada container é deployável de forma independente.

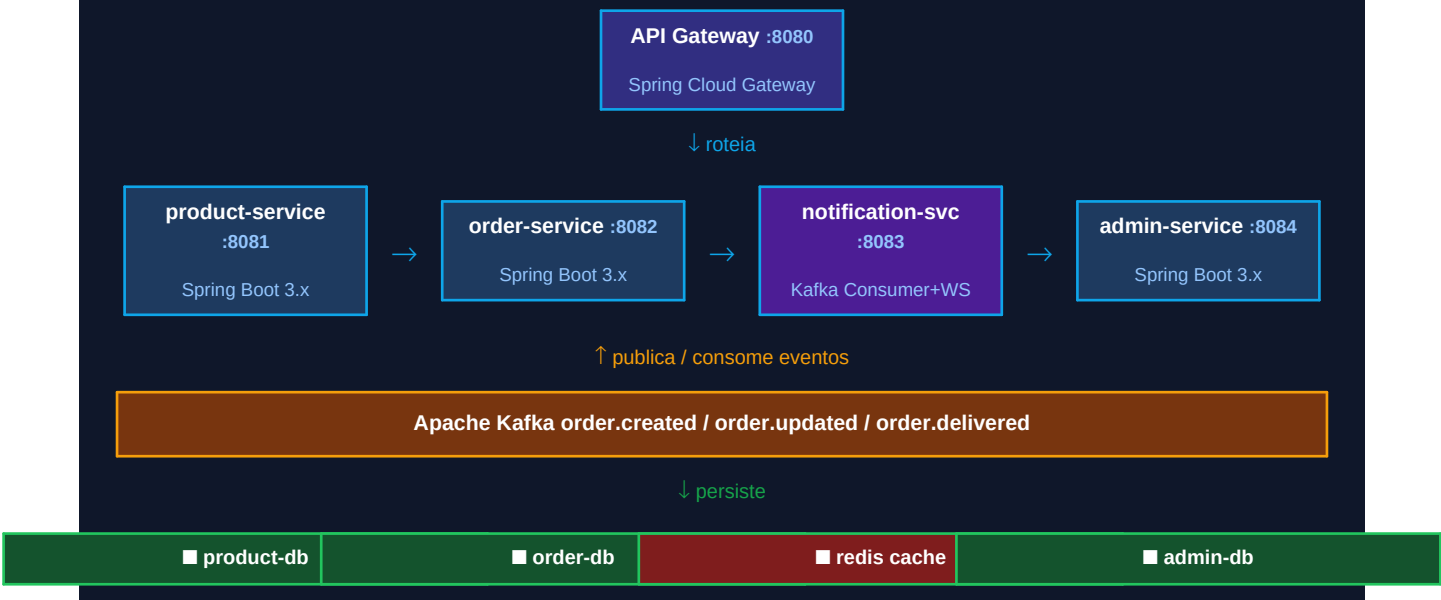


Figura 2 — Diagrama C4 Nível Container: microsserviços, bancos e broker

Descrição dos Containers

Container	Porta	Tecnologia	Responsabilidade
API Gateway	:8080	Spring Cloud Gateway	Ponto de entrada único. Roteia <code>/api/products</code> → <code>product-service</code> , <code>/api/orders</code> → <code>order-service</code> .
product-service	:8081	Spring Boot 3.x	CRUD do catálogo de camisetas. REST puro, sem Kafka. Banco: <code>product-db</code> .
order-service	:8082	Spring Boot 3.x	Cria e gerencia pedidos. Ao criar um pedido publica evento no Kafka. Banco: <code>order-db</code> .
notification-service	:8083	Spring Boot 3.x	Consume eventos Kafka e empurra atualizações ao frontend via WebSocket/STOMP.
admin-service	:8084	Spring Boot 3.x	CRUD de produtos, listagem de pedidos e dashboard de métricas. Banco: <code>admin-db</code> .

4. Diagrama de Classes do Domínio

O diagrama abaixo representa o domínio principal do order-service, que é o núcleo do projeto. Usa recursos modernos do Java 21: sealed interface para modelar a máquina de estados, records imutáveis para DTOs e o padrão Factory para construção do agregado Order.



Figura 3 — Diagrama de Classes: domínio do order-service

Detalhamento das Classes

Classe	Tipo	Responsabilidade
Order	Entity	Agregado raiz do domínio. Contém a lista de OrderItems e o estado atual (sealed interface).
OrderItem	Entity	Item individual do pedido: referência ao produto, quantidade e preço unitário. Calcula subtotal.
OrderStatus	sealed interface	Modela a máquina de estados do pedido com tipos fechados: Processing, Confirmed, Shipped, Delivered, Cancelled.
OrderEvent	record	Payload publicado no Kafka. Imutável por ser record Java 21. Contém orderId, status, email.
OrderFactory	Factory	Centraliza a lógica de criação do agregado Order a partir de um OrderRequest. Aplica validações e calcula total.

Exemplo — Sealed Interface (Java 21)

```
public sealed interface OrderStatus
    permits Processing, Confirmed, Shipped, Delivered, Cancelled {}

// Pattern Matching – o compilador garante todos os casos:
String label = switch (status) {
    case Processing p -> "Processando...";
    case Confirmed c -> "Pedido confirmado!";
```

```
case Shipped s -> "A caminho!";  
case Delivered d -> "Entregue!";  
case Cancelled c -> "Cancelado.";  
};
```

5. Fluxo Assíncrono — Apache Kafka

Este é o fluxo mais importante do projeto e o diferencial técnico exigido pelo documento norteador. O cliente finaliza o pedido, o order-service persiste e publica um evento — sem esperar resposta do notification-service. O frontend é atualizado em tempo real via WebSocket sem recarregar a página.

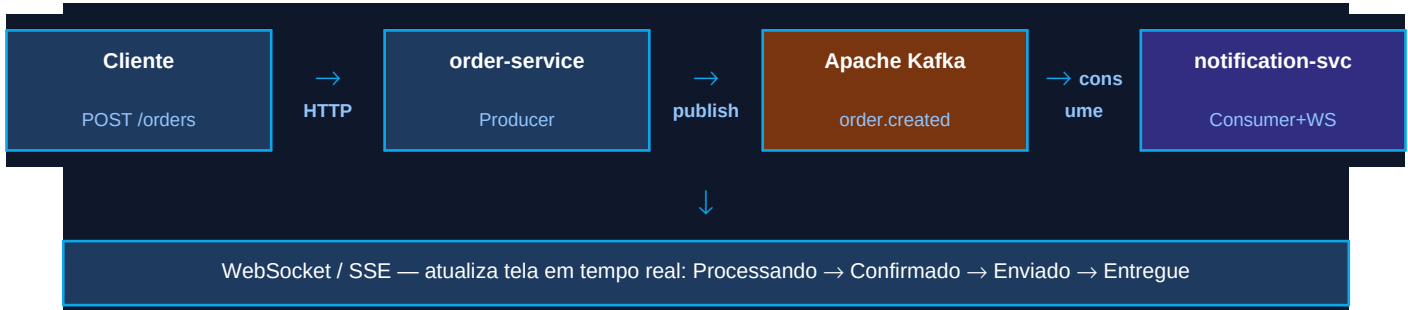


Figura 4 — Fluxo assíncrono: do pedido à atualização da tela em tempo real

Tópicos Kafka

Tópico	Producer	Consumer(s)	Quando é publicado
order.created	order-service	notification-service	Cliente finaliza o carrinho
order.updated	order-service	notification-service	Admin atualiza o status manualmente
order.delivered	order-service	notification-service, admin-service	Pedido marcado como entregue

Configuração Spring Kafka (application.yml)

```
spring:
  kafka:
    bootstrap-servers: kafka:9092
    producer:
      value-serializer: JsonSerializer
      acks: all # garante persistência no broker
    consumer:
      group-id: notification-group
      auto-offset-reset: earliest
      value-deserializer: JsonDeserializer
    properties:
      spring.json.trusted.packages: "com.ecommerce.*"
```


6. Design Patterns Aplicados

Os padrões de projeto foram escolhidos para atender o critério de Qualidade de Código (0,5 pts na N2). Cada padrão resolve um problema concreto do projeto, não foi aplicado apenas para cumprir o requisito.

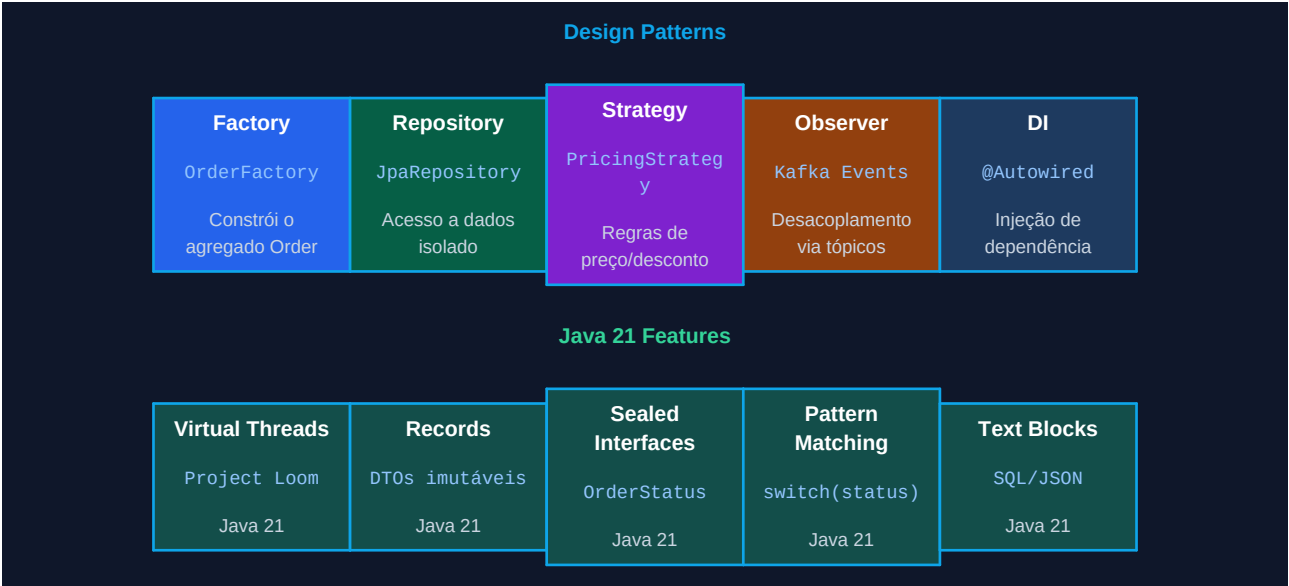


Figura 5 — Design Patterns e features Java 21 utilizados no projeto

Pattern	Implementação	Onde	Justificativa
Factory	OrderFactory	order-service	Centraliza a criação do agregado Order a partir de um OrderRequest.
Repository	JpaRepository	Todos os services	Separa a camada de acesso a dados da lógica de negócio. Facilita mocking.
Strategy	PricingStrategy (interface)	order-service	Encapsula regras de precificação/desconto. Novas regras não alteram o código existente.
Observer	Kafka Topics + @KafkaListener	order / notification	O order-service publica eventos sem conhecer os consumidores. Desacoplamento total.
DTO	Java Records	Controllers	Records imutáveis como DTOs de entrada/saída. Thread-safe por natureza.
DI	@Autowired / Construtor	Todo o projeto	Inversão de controle nativa do Spring. Facilita mocking nos testes unitários.

7. Java 21 — Features Utilizadas

O uso das features modernas do Java 21 demonstra domínio tecnológico e é um argumento técnico forte para a defesa do projeto na banca.

Feature	Onde usar	Benefício
Virtual Threads (Project Loom)	application.yml: spring.threads.virtual.enabled=true	Substitui pool de threads por threads virtuais leves. O servidor suporta muito mais requisições simultâneas com menos memória.
Records	OrderRequest, OrderResponse, OrderEvent, ProductResponse	DTOs imutáveis com zero boilerplate. Construtores, getters, equals, hashCode e toString gerados pelo compilador.
Sealed Interfaces	OrderStatus permits Processing, Confirmed...	Fecha a hierarquia de tipos. O compilador garante que todos os estados são tratados no pattern matching.
Pattern Matching (switch)	switch(status) { case Shipped s -> ...}	Desestrutura e acessa o tipo sem casting manual. Código mais expressivo e seguro.
Text Blocks	SQL nativo em repositórios @Query	Strings multilinhas legíveis para queries SQL complexas e payloads JSON.

Para a banca: habilitar Virtual Threads é uma linha de configuração, mas demonstra conhecimento do ecossistema Java 21. Mencionar que é o Project Loom e que resolve o problema de thread-per-request do modelo bloqueante tradicional gera destaque na arguição.

8. Estrutura dos Repositórios GitHub

O projeto utiliza 5 repositórios na organização GitHub, seguindo o padrão de um repositório por microserviço (exceto o Backend que agrupa product, order e admin como módulos Maven).

Repositório	Conteúdo	Tipo
Projeto-Integrador-Infra	docker-compose.yml, .env.example, README geral da organização	Infra
Projeto-Integrador-Backend	Módulos Maven: product-service, admin-service (Spring Boot 3.x + JPA)	Backend
Projeto-Integrador-Logistics-Service	order-service: criação de pedidos + Kafka Producer	Backend
Projeto-Integrador-Notification-Service	Kafka Consumer + WebSocket/SSE: notificações em tempo real	Backend
Projeto-Integrador-Frontend	React + Vite: vitrine, produto, carrinho, status do pedido, painel admin	Frontend

Estrutura de Pacotes — Padrão por Serviço Spring Boot

```
com.ecommerce.{serviço}/  
  
■■■ controller/ ← @RestController + DTOs de entrada/saída  
  
■■■ service/ ← @Service + lógica de negócio + Strategy  
  
■■■ repository/ ← JpaRepository (acesso a dados)  
  
■■■ messaging/ ← KafkaTemplate (producer) / @KafkaListener (consumer)  
  
■■■ domain/ ← Entidades, Sealed Interface, Factory – zero framework  
  
■■■ dto/ ← Java Records (Request / Response / Event)
```

Regra de ouro da Clean Architecture: o pacote domain/ não pode importar nada do Spring Framework. As entidades e regras de negócio devem ser puro Java, facilitando testes unitários sem subir o contexto Spring (mais rápido, mais simples).

Critérios de Avaliação × Arquitetura

Critério	Como este projeto atende
Coerência da Arquitetura (0,3 N1)	Diagramas C4 + estrutura de microserviços documentada neste documento.
Mensageria desacoplada (0,4 N2)	order-service publica no Kafka sem conhecer o notification-service.
Implementação funcionando (0,4 N2)	Fluxo completo: pedido → Kafka → WebSocket → atualização na tela.
Design Patterns + Arq. Limpa (0,5 N2)	Factory, Repository, Strategy, Observer (Kafka), DI — todos documentados.
Testes (0,4 N2)	JUnit + Mockito (unitários) + Testcontainers (integração com Kafka/PostgreSQL).